

7.14:

Maximum Number of Vertices in VD:

P is a set of n points in the plane, and the farthest point Voronoi diagram is denoted by $FVD(P)$

- Points on convex hull of P can have nonempty farthest point Voronoi cells
- If convex hull has h vertices, then $FVD(P)$ is Voronoi diagram of those h hull vertices, with every cell being unbounded.
- Voronoi diagram for furthest point is a tree

Let h be the number of convex hull vertices of P
In farthest point Voronoi diagram:

- There are h regions, one for each hull vertex
- Every region is unbounded
- Diagram is a tree
- Each leaf of the tree corresponds to an unbounded ray, and there are h leaves.

Let V = number of vertices of the tree, I = internal vertices, $L = h$ = number of leaves, E = number of edges.

- Since it is a tree, $E = V - 1 = I + L - 1$

Use degree sum formula in a tree: $\sum \deg(v) = 2E$

Each leaf has degree 1, and each internal vertex has degree at least 3, so $2E \geq L + 3I$

- Sub in $E = I + L - 1$: $2(I + L - 1) \geq L + 3I$
- It simplifies to: $2I + 2L - 2 \geq L + 3I$
- $L - 2 \geq I$

Therefore $I \leq L - 2 = h - 2$

So number of Voronoi vertices is at most $h - 2$, and since $h \leq n$, the maximum number of vertices possible is $n - 2$ vertices

Farthest Point VD Edges #:

$$E = I + L - 1$$

Maximize E and $I \rightarrow$ already proven that $I \leq h - 2$. So:

$$E \leq (h - 2) + h - 1 = 2h - 3$$

Since $h \leq n \rightarrow E \leq 2n - 3$

Therefore the farthest point Voronoi diagram has at most $2n - 3$ edges

7.19:

Let regions be $R_1, \dots, R_n$ and build the adjacency graph G where two regions are adjacent if an edge is shared.

- If R_i and R_j are Voronoi cells sharing an edge e , then the line that supports e is the perpendicular bisector of the two sites p_i, p_j . So p_j is the reflection of p_i across that line.

Algorithm:

1. Build the adjacency graph of the subdivision.
2. For each shared edge (i, j) , calculate the reflection σ_{ij} across its supporting line.
3. Choose one region R_r as a root. Its site $p_r = (x, y)$ is unknown and pick a spanning tree of G
4. Traverse the tree. If j is a child of i , set $p_j = \sigma_{ij}(p_i)$
 - a. All sites then become an affine function of (x, y)
5. For every non tree edge (i, j) , consistency $p_j = \sigma_{ij}(p_i)$
 - a. This gives linear equations in x, y form. If system is inconsistent, the subdivision is not a Voronoi diagram
6. Otherwise, compute all candidate sites and construct their Voronoi diagram.
 - a. If it matches the subdivision, output the sites. Otherwise no such set exists.

This works because adjacent cells in VD must satisfy the reflection condition, so any valid set of sites satisfies the equations. If recovered sites produce the same subdivision, then they are desired sites.

Runtime: Building constraints = $O(n)$, constructing and checking VD = $O(n \log n)$

Total = $O(n \log n)$

9.2:

Claim: In every triangulation of point set, the interior point p has degree $n - 1$

Example:

Take $n - 1$ points on a circle and place one point p inside the circle.

Let the outer points be $q_1, \dots, q_{n-1}$ in circular order.

Since all q_i lie on the convex hull, the triangulation decomposes the convex $(n - 1)$ sided polygon into triangles.

The interior point p lies inside the polygon, so for any triangulation it must be connected to the vertices of the triangle that contains it.

Repeat this around p , the only way to correctly triangulate the polygon without crossing any edges is to connect p to every hull vertex.

Triangles around p form a fan, so all edges pq_i must be present

Therefore, p is adjacent to all other $n - 1$ points.

So degree = $n - 1$

Therefore this point set proves through an example that the claim is true.

9.6:

Algorithm:

Step 1:

Keep and maintain a stack of edges that may need to be legalized

Step 2:

After inserting new point p_r , the three edges opposite p_r in the new triangles are pushed onto the stack

Step 3:

While the stack is nonempty, pop an edge e

Step 4:

If e is on the convex hull, it can not be flipped, so do nothing

Step 5:

Otherwise, test if e is illegal using the incircle test

Step 6:

If e is legal then do nothing.

Otherwise if e is illegal, then flip it

Step 7:

After flip, the only edges that become illegal are the two end edges opposite p_r , so push them onto the stack

Step 8:

When the stack becomes empty, all relevant edges are legal. Then the triangulation is Delaunay again.

ITERATIVE-LEGALIZEEDGE(T, S, pr)

Input T = current triangulation, S = stack of new edges, pr = new point that was just inserted.

```

while S is nonempty:
  pop edge e = (pi, pj) from S
  If e is a hull edge then continue
  let pk = vertex opposite e in triangle on one side
  let pl = vertex opposite e in adjacent triangle on the other
  if pl is inside circumcircle of triangle then
    flip e
    let e1 = (pi, pj)
    let e2 = (pj, pl)
    push e1 onto S
    push e2 onto S
  end if
end while

```

Advantages/Disadvantages:

The iterative version of the algorithm has more control over memory and avoids the stack overflowing if too many edges flip. The disadvantages of the iterative algorithm is that the stack must be maintained, edges can be possibly pushed more than once.

9.8:

Let $\triangle p_i p_j p_k$ be a Delaunay triangle of P that contains q

Show that after inserting q , the edges qp_i , qp_j , and qp_k all belong to the Delaunay triangulation of $P \cup \{q\}$

Since $\triangle p_i p_j p_k$ is Delaunay triangle of P , its circumcircle contains no point of P in its interior.

Let C be the circumcircle. Since q lies inside triangle $\triangle p_i p_j p_k$ it also lies inside the circumcircle C .

Need to show triangle is Delaunay in $P \cup \{q\}$.

Since q lies inside $\triangle p_i p_j p_k$, the circumcircle through q, p_j, p_k is inside of the circumcircle C of p_i, p_j, p_k on the side of edge $p_i p_j$ containing q .

Replacing p_k by interior point q makes circumcircle smaller.

So all points inside C_{ij} are also inside C . Since C contains no point of P inside of it, C_{ij} also has no points of P in its interior.

q lies on the boundary of C_{ij} , so C_{ij} contains no points in $P \cup \{q\}$ in its interior besides defining vertices.

Therefore $\triangle qp_i p_j$ satisfies the empty circle property, so it is a Delaunay triangle of $P \cup \{q\}$.

This also applies to the $\triangle qp_j p_k$ and $\triangle qp_k p_i$ triangles. Circumcircles also do not contain points of $P \cup \{q\}$. So all three triangles are Delaunay triangles in $P \cup \{q\}$.

Since each of the triangles are in the Delaunay triangulation of $P \cup \{q\}$. The edges qp_i , qp_j , and qp_k must be Delaunay edges.

Therefore, if q lies inside a Delaunay triangle $p_i p_j p_k$, then after inserting q , it is connected to all three vertices of the triangle in the Delaunay triangulation of $P \cup \{q\}$.

qp_i , qp_j , and qp_k are edges of $\text{DT}(P \cup \{q\})$